

TP commande d'un système sur la base d'un modèle appris

Florent Bergé
Léa Grousseau

28 mars 2026

1 Introduction

L'objectif pédagogique de ce TP est de découvrir la dynamique d'un système avec un réseau de neurones en faisant la synthèse d'un contrôleur sur la base du modèle appris.

2 Présentation du système et modélisation

Nous étudions le système MIMO (Multiple Inputs Multiple Outputs) suivant :

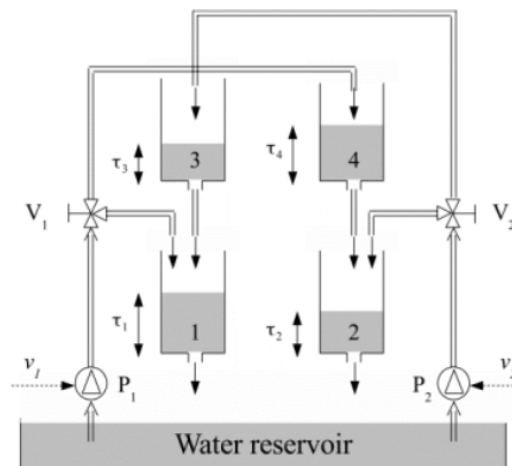


FIGURE 1 – Schéma du système

L'objectif est d'asservir la hauteur d'eau des réservoirs 1 et 2 grâce à la commande des 2 pompes. Pour ce faire, nous allons identifier le système à l'aide de réseaux de neurones.

2.1 structure des fichiers

Les fichiers rédigés pour ce TP sont :

- *INIT_data.m* : prépare les données ;
- *INIT_mlp.m* : entraîne les 6 modèles de régression de la partie 4.1 ;
- *INIT_modele_structure.m* : entraîne le modèle structuré ;
- *trained_models.mat* : fichier dans lequel les modèles sont enregistrés une fois entraînés ;
- *INIT_lq.m* : permet de linéariser les modèles à un point d'équilibre et résoudre un problème LQR (pour les MLPs et le modèle structuré) ;

3 Exploration des données d'apprentissage

3.1. Le réseau sera un modèle de régression, basé sur des données fournies (en figure 2). Le critère à minimiser adéquat est la fonction Mean Square Error (MSE).

3.2. Nous prétraitons les données comme suit :

```
load("data.mat");
import casadi.*

%% Initialisation des donnees
% On cherche a identifier f tq  $x_{k+1} = f(x_k, u_k)$ 

y = states(:, 2:end)./xmax; %  $x_{k+1}$ 
x = [states(:,1:end-1)./xmax; inputs./umax]; % [ $x_k$ ;  $u_k$ ]
```

Ici, y est la sortie du modèle, correspondant à x_{k+1} , et x est l'entrée, composée de x_k et u_k .

3.3. En traçant les données inputs on observe que les commandes pour les pompes 1 et 2 varient beaucoup, entre 0 et 0.0011. Les données de states montrent que les niveaux d'eau des 4 récipients varient aussi entre 0 et 1.3 cette fois. (figure 2)

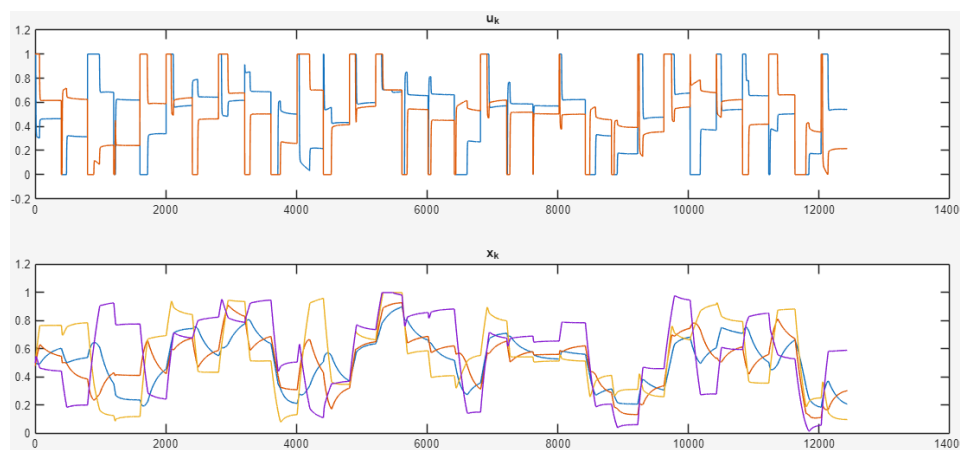


FIGURE 2 – Affichage des données d'entraînement normalisées

Pour les deux jeux de données, ces variations se font « par bloc », indépendamment des autres états et commandes, de façon presque aléatoire.

3.4. Nous mélangeons ensuite les données et séparons les données d'entraînement et de validation :

```
% Melange des donnees
permutation = randperm(size(x,2));
xperm      = x(:, permutation); yperm= y(:, permutation);

% Separation des donnees 70/30
i_test     = floor(0.7*size(x,2));
x_train    = xperm(:, 1:i_test);
y_train    = yperm(:, 1:i_test);
```

```
x_valid = xperm(:,i_test+1:end);
y_valid = yperm(:,i_test+1:end);
```

4 Apprentissage de la dynamique du système

4.1 Perception multicouche

4.1.1. Le réseau comporte une couche d'entrée de taille $n_{in} = 6$, une couche cachée de taille n_{hidden} , une couche de sortie de taille $n_{out} = 4$ et une fonction d'activation (ReLU, Tanh ou Sigmoid). Nous initialisons notre modèle comme suit :

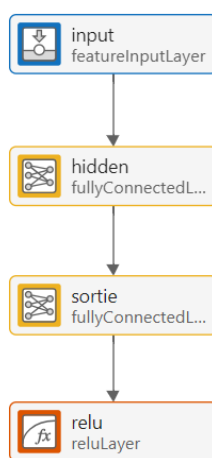


FIGURE 3 – Schéma du modèle

Dans les réseaux de neurones, il y a un biais par neurones et un poids par liaison. Dans le modèle, il y a $n_{hidden} + 4$ neurones et $6n_{hidden} + 4n_{hidden}$ liaisons, soit un total de $11n_{hidden} + 4$ paramètres.

4.1.2. Il suit dans Matlab :

```
%% Initialisation du modèle
n_in      = size(x,1);
n_hidden  = 12;
n_out     = size(y,1);
net = dlnetwork;
tempNet = [
    featureInputLayer(n_in,"Name","input")
    fullyConnectedLayer(n_hidden,"Name","hidden_layer")
    fullyConnectedLayer(n_out,"Name","output_layer")];
net_tanh_0 = addLayers(net,[tempNet; tanhLayer("Name","tanh")]);
net_relu_0 = addLayers(net,[tempNet; reluLayer("Name","relu")]);
net_sigmoid_0 = addLayers(net,[tempNet; sigmoidLayer("Name", ...
"sigmoid")]);

% clean up
clear tempNet;
```

```
net_relu = initialize(net_relu_0);
net_tanh = initialize(net_tanh_0);
net_sigmoid = initialize(net_sigmoid_0);
```

4.1.2.a. Comme dit précédemment, nous utilisons la fonction MSE comme fonction de perte, soit :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

4.1.2.b. Pour mettre en place la boucle d'entraînement, nous définissons la fonction de perte :

```
function [loss, gradients] = lossFunction(model, x, y)
    % Forward pass
    y_hat = forward(model, darray(x, 'CB'));

    % Compute MSE loss
    loss = mse(y_hat, darray(y, 'CB'));

    % Compute gradients
    gradients = dlgradient(loss, model.Learnables);
end
```

Nous définissons ensuite la boucle d'entraînement avec la méthode de descente du gradient dans une fonction matlab également :

```
function net = gradientTraining(net, nb_batch, batch_size, nb_epochs, eta, x, y)
for epoch = 1:nb_epochs
    epoch_loss = 0;
    for i = 1:nb_batch
        idx = (i-1)*batch_size + 1 : i*batch_size;
        x_batch = darray(x(:, idx), 'CB');
        y_batch = darray(y(:, idx), 'CB');

        [loss, gradients_relu] = dlfeval(@lossFunction, ...
            net, x_batch, y_batch);
        net = dlupdate(@(w,g) w - eta*g, net, gradients_relu);
        epoch_loss = epoch_loss + extractdata(loss);
    end
    fprintf('Epoch %d/%d - Loss: %.6f\n', epoch, ...
        nb_epochs, epoch_loss/nb_batch);
end
```

4.1.3. Nous souhaitons comparer les différentes fonctions d'activation. Le code ci-dessous permet de lancer l'entraînement pour chaque modèle :

```
%% Initialisation de l'entrainement
nb_batch = 113; % x_train size is 8701 = 7x11x113
batch_size = 77; % Taille d'un batch
eta = 1e-3; % Learning rate
nb_epochs = 20; % Nombre d'iterations d'entrainement

% Entrainement
fprintf("\n==== Relu ====\n")
```

```

net_relu = gradientTraining(net_relu, nb_batch, ...
    batch_size, nb_epochs, eta, x_train, y_train);
fprintf("\n==== Tanh ====\n")
net_tanh = gradientTraining(net_tanh, nb_batch,...
    batch_size, nb_epochs, eta, x_train, y_train);
fprintf("\n=== Sigmoid ===\n")
net_sigmoid = gradientTraining(net_sigmoid, ...
    nb_batch, batch_size, nb_epochs, eta, x_train, y_train);

```

Comparons la valeur de la fonction de perte à la fin de l'entraînement pour 20 epochs.

Fonction d'activation	$n_{hidden} = 8$	$n_{hidden} = 12$
ReLu	0.166276	0.015629
Tanh	0.048531	0.064516
Sigmoïd	0.079691	0.069873

TABLE 1 – Valeur de la fonction perte à la fin de l'entraînement

Il ressort que les fonctions d'activations les plus efficaces sont dans l'ordre ReLu, Tanh puis Sigmoïd et que $n_{hidden} = 12$ est le plus performant. On choisit donc de poursuivre l'étude avec 12 neurones dans la couche cachée.

4.1.4. Changeons maintenant la descente du gradient pour l'algorithme *adam* :

```

function net = adamTraining(net, nb_batch, batch_size, nb_epochs, eta, x, y)
    avg_grad = [];
    avg_sq_grad = [];
    for epoch = 1:nb_epochs
        epoch_loss = 0;
        for i = 1:nb_batch
            idx = (i-1)*batch_size + 1 : i*batch_size;
            x_batch = dldarray(x(:, idx), 'CB');
            y_batch = dldarray(y(:, idx), 'CB');

            [loss, gradients] = dlfeval(@lossFunction, net, ...
                x_batch, y_batch);
            [net, avg_grad, avg_sq_grad] = adamupdate(net, ...
                gradients, avg_grad, avg_sq_grad, ...
                epoch+i-1, eta);
            epoch_loss = epoch_loss + extractdata(loss);
        end

        fprintf('Epoch %d/%d - Loss: %.6f\n', epoch, ...
            nb_epochs, epoch_loss/nb_batch);
    end
end

```

4.1.5. Nous obtenons les résultats suivants, pour $n_{hidden} = 12$:

Fonction d'activation	<i>gradient</i>	<i>adam</i>
ReLu	0.015629	0.002551
Tanh	0.064516	0.012651
Sigmoïd	0.069873	0.003141

TABLE 2 – Valeur de la fonction perte à la fin de l'entraînement

Les résultats sont bien plus satisfaisants avec la mise à jour des paramètres, et cette fois c'est la fonction d'activation ReLu qui est la plus performante.

4.1.6. Nous calculons les fonctions MSE, RMSE et R^2 pour le modèle ReLu entraîné avec *adam* :

```

===== Validation Results =====
State 1 | MSE: 0.000062 | RMSE: 0.007843 | R2: 0.9980
State 2 | MSE: 0.000188 | RMSE: 0.013697 | R2: 0.9944
State 3 | MSE: 0.000526 | RMSE: 0.022944 | R2: 0.9917
State 4 | MSE: 0.000667 | RMSE: 0.025824 | R2: 0.9905
=====
Mean R2   : 0.9936
Mean RMSE : 0.017577

```

La moyenne de R^2 est proche de 1 et celle de la racine de notre fonction de coût (RMSE) est faible ce qui montre que ce modèle est adapté à notre simulation.

4.2 Approche structurée

Nous nous sommes donné pour objectif d'apprendre la dynamique discrétisée du système. Nous pourrions tout à fait décider d'apprendre la dynamique à temps continu à la place, c'est à dire apprendre le modèle :

$$\dot{x}(t) = f(x(t), u(t))$$

Seulement, nous ne disposons que de données à temps discret et non pas à temps continu du système. Par ailleurs, les commandes sont bloquées pendant 1 seconde, puisque le contrôleur qui a généré les données est un contrôleur à temps discret. Tout d'abord, nous faisons l'hypothèse que le système est en fait linéaire vis-à-vis des entrées. On peut donc réécrire la dynamique sous la forme :

$$\dot{x} = g(x) + Bu$$

Supposons que l'on intègre numériquement ce système avec un schéma d'Euler, on a alors :

$$x_{k+1} = x_k + h\dot{x}(t_k) = x_k + hg(x_k) + hBu_k$$

Nous allons procéder à l'apprentissage structuré de la dynamique du système, sous hypothèse de linéarité vis-à-vis de la commande et en cherchant la dynamique continue du système, c'est à dire un réseau $R_\theta(x_k, u_k)$ qui implémente la structure présentée en figure 4.

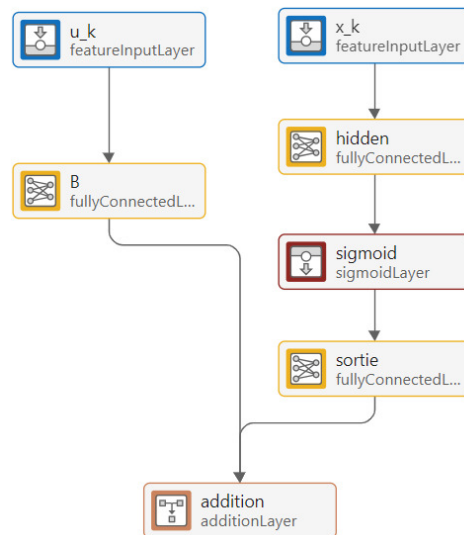


FIGURE 4 – Schéma du modèle structuré

4.2.2. La partie du réseau qui apprend la dynamique continue est la couche B composée de 4 neurones de sortie se trouvant après l'entrée de la commande u_k .

4.2.3. Ce type de réseau est appelé "réseau structuré".

Pour l'implémentation du réseau, nous choisissons une taille $n_{hidden} = 12$, une fonction d'activation Sigmoid et l'utilisation de l'algorithme *adam* pour ce modèle. Dans Matlab :

```
%% Initialisation du modele
n_hidden    = 12;
n_out       = size(y,1);
net         = dlnetwork;

tempNet = [
    featureInputLayer(4,"Name","x_k")
    fullyConnectedLayer(n_hidden,"Name","hidden")
    sigmoidLayer("Name","sigmoid")
    fullyConnectedLayer(n_out,"Name","sortie")];
net = addLayers(net,tempNet);

tempNet = [
    featureInputLayer(2,"Name","u_k")
    fullyConnectedLayer(n_out,"Name","B")];
net = addLayers(net,tempNet);

tempNet = additionLayer(2,"Name","addition");
net = addLayers(net,tempNet);

% clean up helper variable
clear tempNet;
```

```

net = connectLayers(net,"sortie","addition/in1");
net = connectLayers(net,"B","addition/in2");
net_struct = initialize(net);

% nouvelle fonction forward
function y_hat = forwardStruct(model, x_k, u_k)
    % x_k+1 = x_k + h*g(x_k) + h*B*u_k
    h = 0.01;
    y_hat = x_k + h * forward(model, x_k, u_k);
end

```

4.2.4. et 4.2.5. Nous entraînons ce modèle avec l'algorithme *adam* et les mêmes paramètres que précédemment. nous obtenons finalement :

```

===== Validation Results =====
State 1 | MSE: 0.000007 | RMSE: 0.002732 | R2: 0.9998
State 2 | MSE: 0.000003 | RMSE: 0.001661 | R2: 0.9999
State 3 | MSE: 0.000003 | RMSE: 0.001783 | R2: 1.0000
State 4 | MSE: 0.000002 | RMSE: 0.001470 | R2: 1.0000
=====
Mean R2   : 0.9999
Mean RMSE: 0.001912

```

La mesure liée à notre fonction de coût (RMSE) est encore plus faible pour le réseau structuré que pour le réseau MLP et celle liée à la mesure des résidus (R^2) est encore plus proche de 1. Ce réseau est donc plus performant pour la simulation.

5 synthèse d'un contrôleur LQ

5.1. Nous souhaitons désormais utiliser le modèle pour synthétiser un contrôleur LQ. Pour cela, il nous faut un modèle linéaire. Nous allons linéariser le modèle structuré autour d'un point d'équilibre noté (x_{ref}, u_{ref}) . Notons qu'à ce point $f(x_{ref}, u_{ref}) = 0$.

$$f(x, u) = \frac{\partial f}{\partial x(x_{ref}, u_{ref})} (x - x_{ref}) + \frac{\partial f}{\partial u(x_{ref}, u_{ref})} (u - u_{ref})$$

5.2. Nous prenons comme point de référence :

```

%% Point de reference
x_ref = [0.6; 0.8; 0.4334; 1.1449];
u_ref = 1e-4*[5.972; 4.505];
input = dlarray([x_ref./xmax; u_ref./umax], 'CB');

```

Le code suivant permet de déterminer les matrices A et B à l'aide des matrices jacobiniennes. Nous utilisons *dljacobian*, fonction qui doit être utilisée dans une exécution de *dlfeval* :

```

%% Calcul des jacobiniennes
function [y_hat, jacX, jacU] = jacobian(model, x_k)
    y_hat = forward(model, dlarray(x_k, 'CB'));
    jac   = dljacobian(stripdims(y_hat), stripdims(x_k), 1);

```

```

    jacX = jac(:,1:4);
    jacU = jac(:,5:6);
end

function [y_hat, jacX, jacU] = jacobian_struct(model, x_k, u_k)
    y_hat = forwardStruct(model, x_k, u_k);
    jacX = dljacobian(stripdims(y_hat), stripdims(x_k), 1);
    jacU = dljacobian(stripdims(y_hat), stripdims(u_k), 1);
end

% --- Jacobienne ---
[~, dJdx, dJdu] = dlfeval(@jacobian, net_sigmoid_adam, input);
%[~, dJdx, dJdu] = dlfeval(@jacobian_struct, net_struct, ...
    input(1:4,:), input(5:6,:));
A = extractdata(dJdx);
B_lin = extractdata(dJdu);

```

5.3. pour la synthèse du contrôleur LQ nous utilisons la règle de Bryson :

```

%% Calcul du gain LQ

% Matrices de ponderation de Bryson
Q = diag(1 ./ (xmax.*xmax)); % [4x4]
R = diag(1 ./ (umax.*umax)); % [2x2]

% Calcul du gain LQR (systeme discret)
[K, S, P] = dlqr(A, B_lin, Q, R);

abs(eig(A - B_lin * K)) % <1

```

5.4. Nous implémentons ensuite la boucle de simulation suivante.

```

%% Simulation en boucle sur le modele reseau de neurones

N      = 3000;
x_sim  = zeros(4, N+1);
u_sim  = zeros(2, N);
x_sim(:,1) = 0.65*ones(4,1); % condition initiale

for k = 1:N
    % Loi de commande LQR
    % Normalisation des etats puis 'denormalisation' de u
    u_sim(:,k) = (u_ref./umax - K * (x_sim(:,k)./xmax - ...
        x_ref./xmax)).*umax;

    % Saturation des entrees
    u_sim(:,k) = max([0; 0], min(umax, u_sim(:,k)));

    % Fonction casadi
    x_next = phi(x_sim(:,k), u_sim(:,k));
    x_sim(:,k+1) = full(x_next);
end

% Affichage

```

```

t = (0:N) * h;
figure
subplot(2,1,1)
plot(t, x_sim')
legend('x1','x2','x3','x4')
xlabel('Temps (s)'); ylabel('Etats')
title('Etats en boucle fermée (casadi)')
yline(x_ref', '--', {'x1_{ref}','x2_{ref}','x3_{ref}','x4_{ref}}')

subplot(2,1,2)
plot(t(1:end-1), u_sim')
legend('u1','u2')
xlabel('Temps (s)'); ylabel('Commandes')
title('Commandes LQR')

```

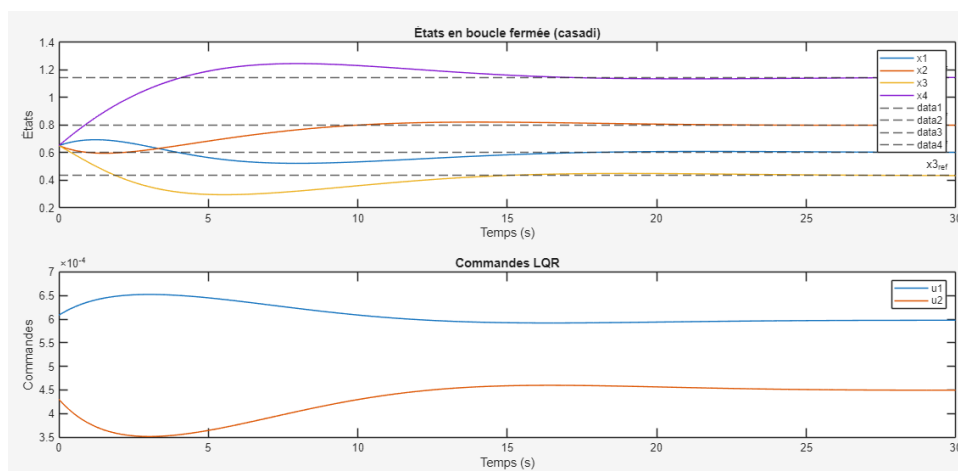


FIGURE 5 – structuré

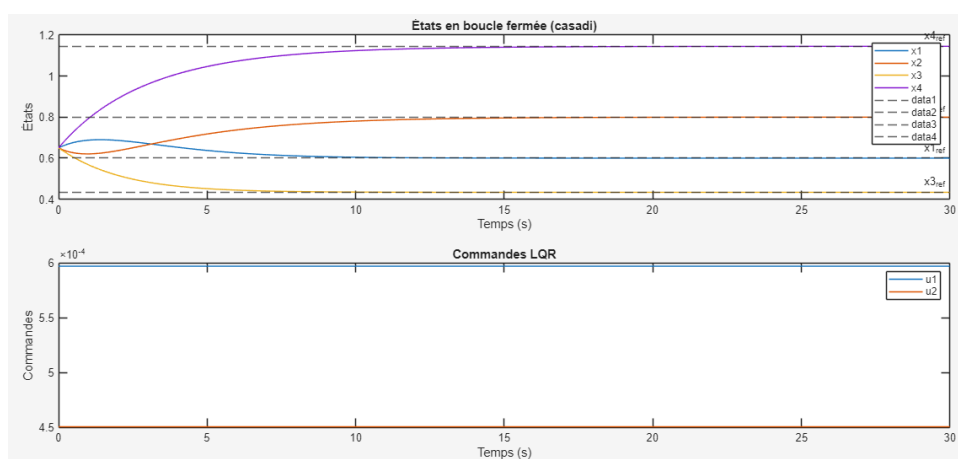


FIGURE 6 – relu

On observe que les régulations faites avec les contrôleurs obtenus par le réseau structuré ou par la fonction d'activation Relu convergent toutes les deux vers les valeurs de références attendues en régimes permanent (les "data1, data2,

data3 et data4). Dans le cas du réseau structuré, on observe de légers dépassements sur les 15 premières secondes. Pour le cas de la fonction d'activation Relu, les commandes restent constantes aux valeurs de u_{ref} .

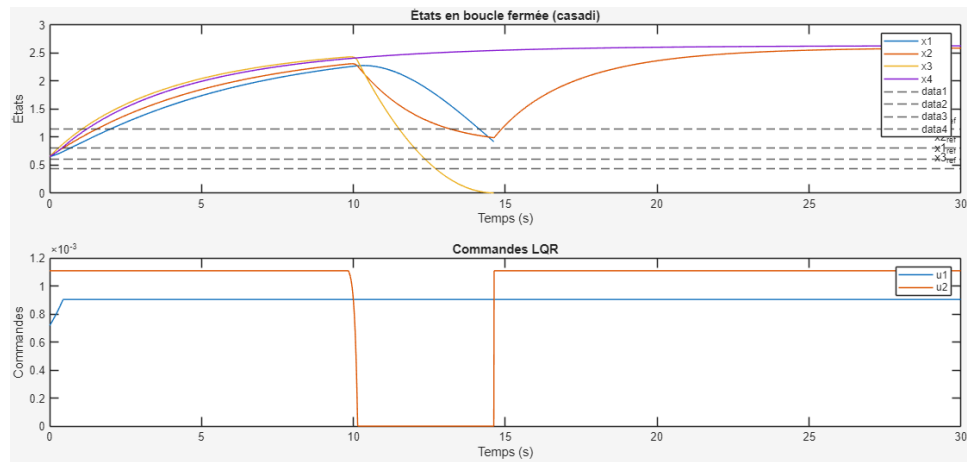


FIGURE 7 – tanh

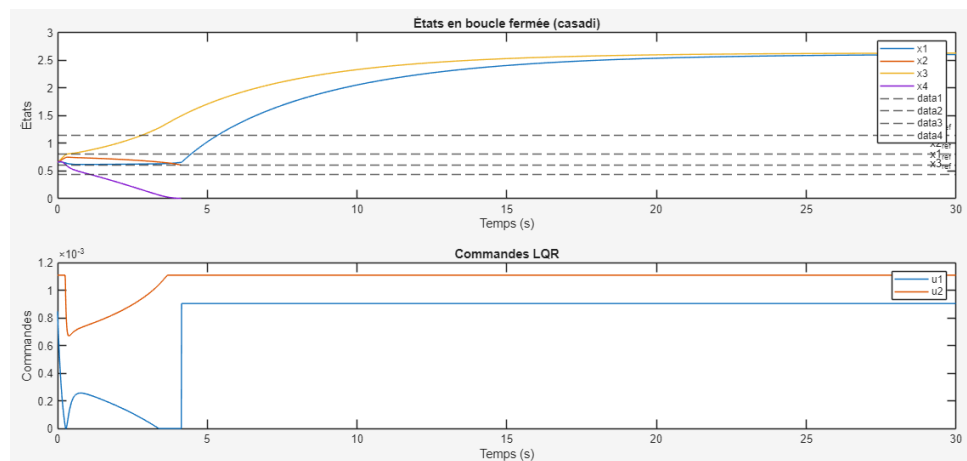


FIGURE 8 – sigmoid

En revanche, pour ce qui est les contrôleurs obtenus par les fonctions d'activation tanh et sigmoïd ne sont pas bons.